



YLang

COMPILED · EXPLICIT · FRIENDLY

The YLang Book

A beginner-friendly guide to YLang 1.1.0

Edition for YLang 1.1.0

Contents

Chapter 1. What is YLang?	2
Chapter 2. Installing YLang	3
Chapter 3. Your First Program	5
Chapter 4. The Commands	7
Chapter 5. Variables and Constants	8
Chapter 6. Types	10
Chapter 7. Printing and F-Strings	12
Chapter 8. Operators	14
Chapter 9. Making Decisions with if	15
Chapter 10. Repeating Things: loop, while, for	16
Chapter 11. Functions	19
Chapter 12. Arrays	20
Chapter 13. Converting Between Types	23
Chapter 14. The Math Library	25
Chapter 15. Strings, Paths and Input/Output	27
Chapter 16. Reading Errors and Using fix	31
Chapter 17. Runtime Errors	33
Chapter 18. A Complete Program	35
Chapter 19. Optional Extras	37
Chapter 20. What YLang Does Not Have	38
Chapter 21. Quick Reference	39

Every code sample in this book was compiled and run with YLang 1.1.0, so the outputs you see are the outputs you will get. The exceptions are the GUI runtime and the VS Code extension in Chapter 19, and the success path of image.open in Chapter 15, which need a desktop that my test machine does not have.

Chapter 1. What is YLang?

YLang is a small programming language. You write your program in a text file that ends with `.yl`. Then the YLang compiler checks your program and turns it into a real program your computer can run.

Here is what makes YLang special:

- **It is compiled.** There is no interpreter. YLang translates your code to C, then uses GCC or Clang to build a native executable.
- **Types are explicit.** You always say what kind of value a variable holds (`int`, `string`, and so on).
- **Errors are friendly.** Every error has a code (like `E2001`), the exact line and column, a picture of your code with an arrow, and a hint.
- **It is small on purpose.** It has a short list of features, and it does them carefully.

What you can do with it today:

- Variables, constants, and five basic types, with explicit conversions between them
- `if / else`, and three kinds of loops: `loop()`, `while`, and `for`
- Shortcuts like `count += 1` and `count++`
- Functions
- Fixed-size arrays with bounds checking, which can be passed to functions
- A `math` library, a `string` library, a `path` library, and an `io` library for reading input and text files
- An optional GUI runtime (for C code, see Chapter 19)

YLang is written in C and released under the MIT License. The project lives at github.com/Yassine-Jemi01/ylang.

Version. This edition of the book covers YLang **1.1.0**. Compared with 1.0.0 it adds type conversions, checked number parsing, passing arrays to functions, `length()`, `+=` and `++`, hexadecimal and scientific number literals, block comments, string ordering, more string and path helpers, and a reference of error codes.

Chapter 2. Installing YLang

YLang is distributed as source code. You build the compiler once on your machine, and then you can use it forever.

2.1 What you need

- **Linux** (Fedora and Ubuntu are tested) or **macOS 13 or newer**.
- **Windows is not supported yet.** The compiler is made for POSIX systems. A common workaround is WSL (Windows Subsystem for Linux), where you follow the Linux steps. That is my suggestion, not an official promise from the YLang docs.
- **A C compiler:** `gcc` or `clang`. You need it twice: once to build YLang itself, and again every time YLang builds your programs.
- `make`, the build tool.

Install them on **Ubuntu or Debian**:

```
sudo apt install build-essential clang make
```

Install them on **Fedora**:

```
sudo dnf install gcc make clang
```

Install them on **macOS**:

```
xcode-select --install  
brew install make llvm
```

On macOS, type `gmake` instead of `make` in every command in this book. The built-in macOS `make` is too old.

2.2 Get the source

Option A: you have the zip file (`ylang-main.zip`). Unzip it and go inside the folder:

```
unzip ylang-main.zip  
cd ylang-main
```

Option B: download it with git:

```
git clone https://github.com/Yassine-Jemi01/ylang.git  
cd ylang
```

2.3 Build the compiler

```
make
```

This creates the compiler at `build/ylang`.

2.4 Run the tests (recommended)

```
make test
```

If everything is fine, the last line says:

```
All YLang tests passed.
```

2.5 Check that it works

```
./build/ylang --version
```

Expected output:

```
YLang compiler 1.1.0 (C backend: GCC/Clang)
```

2.6 Install it system-wide (optional)

Right now you must type `./build/ylang` every time. If you want to type just `ylang` from any folder, install it:

```
sudo make install
```

This copies it to `/usr/local/bin/ylang`. To install in your home folder without `sudo`:

```
make install PREFIX="$HOME/.local"
```

(Make sure `$HOME/.local/bin` is in your `PATH`.) To remove it later:

```
sudo make uninstall
```

2.7 Updating

When a new version of the source comes out, get the new files (`git pull`, or unzip the new zip), then run `make clean`, `make`, and `make test` again.

In the rest of this book, I write `ylang` for the command. If you did not install it, write `./build/ylang` instead.

Chapter 3. Your First Program

Create a file called `hello.yl`:

```
function main() -> int {  
    print("Hello, YLang!");  
    return 0;  
}
```

Now build it and run it:

```
ylang build hello.yl -o hello  
./hello
```

Output:

```
Hello, YLang!
```

Let us read the program line by line:

- `function main() -> int {` starts a function named `main`. Every YLang program must have one. It is where the program starts. `-> int` means it gives back an integer when it finishes.
- `print("Hello, YLang!");` shows text on the screen.
- `return 0;` ends the program. `0` means success.
- `}` closes the function.

Two rules you will see everywhere:

- 1 Every statement ends with a semicolon `;`.**
- 2 Blocks of code live inside curly braces `{ }`.**

Comments start with `//` and go to the end of the line:

```
// This is a comment. YLang ignores it.  
print("Hi"); // This is also a comment.
```

Comments can also run over several lines. They start with `/*` and end with `*/`:

```
/* This comment  
   uses two lines. */
```

Block comments cannot be nested, and you must close them. If you forget the `*/`, YLang says so:

```
error[E1001]: Unterminated block comment; expected '*/'.
```

The main function

`main` can have two forms. Both take no parameters:

```
function main() -> int { return 0; }
```

```
function main() -> void { }
```

`void` means "returns nothing".

Chapter 4. The Commands

The `ylang` tool has four commands you will use.

Command	What it does
<code>ylang check file.yl</code>	Checks your program for mistakes. Does not make an executable. Fast.
<code>ylang build file.yl -o name</code>	Checks your program, then makes an executable called <code>name</code> .
<code>ylang emit-c file.yl -o out.c</code>	Saves the generated C code so you can read it.
<code>ylang fix file.yl -o fixed.yl</code>	Fixes one specific typo automatically (see Chapter 16).

Other useful things:

- `ylang --help` shows help.
- `ylang --version` shows the version.
- `--cc clang` (or `--cc gcc`) chooses which C compiler `build` uses:

```
ylang build hello.yl -o hello --cc clang
```

If your program has errors, `build` will **not** create an executable. That is a feature: you never run a half-checked program.

Speed settings

By default, YLang asks the C compiler for the `-O2` optimization level. You can change it with the `YLANG_OPT_LEVEL` environment variable. The allowed values are `0`, `1`, `2`, `3`, and `s`:

```
YLANG_OPT_LEVEL=3 ylang build app.yl -o app
```

Use `0` when you are debugging, `2` for a good balance (the default), `3` to test maximum speed, and `s` for smaller programs. A wrong value is rejected:

```
ylang: invalid YLANG_OPT_LEVEL '9' (use 0, 1, 2, 3, or s)
```

Clean builds

Earlier versions sometimes showed C compiler warnings about the generated code (for example, an unused variable). In 1.1.0 the common unused-variable warnings are hidden in normal builds, so a build that works prints just `Build succeeded`.

Chapter 5. Variables and Constants

A **variable** is a named box that holds a value. In YLang you create one with `let`, then the **type**, then the **name**:

```
let int score = 100;
let string name = "YLang";
let bool enabled = true;
```

You can also create a variable without a value and give it one later:

```
let int age;
age = 16;
```

To change a variable, use `=`:

```
score = score + 10;
```

Constants

A **constant** is a variable that can never change. Add `const` after `let`:

```
let const int LIMIT = 100;
```

A constant must be given a value right away. If you try to change it, YLang stops you:

```
function main() -> int {
    let const int MAX = 100;
    MAX = 200;
    return 0;
}
```

```
error[E2022]: Cannot assign to a const variable.
--> tests/const-assignment.yl:3:9
  |
3 |     MAX = 200;
  |         ^
= help: Keep 'const' and remove this assignment, or remove
'const' if the variable should be mutable.
```

The "initialized before reading" rule

YLang will not let you read a local variable unless it is sure the variable has a value on every possible path:

```
function main() -> int {
    let int score;
    if (true) {
        score = 10;
    }
    print(score);
    return 0;
}
```

```
error[E2021]: This variable may be used before it is initialized.
--> examples/uninitialized.yl:6:11
    |
  6 |     print(score);
    |           ^^^^^
    = help: Assign a value to 'score' before reading it.
```

The fix is to give `score` a value in an `else` branch too, or to give it a starting value when you declare it. This version is accepted, because both paths assign `score`:

```
let int score;
if (true) {
    score = 10;
} else {
    score = 20;
}
print(score);
```

Global variables

Variables declared outside any function are **global**. A global variable without a value gets a safe default: `0` for numbers, `""` for strings, `false` for booleans. Still, it is better to write the value yourself.

```
let string blank;
let int counter;

function main() -> int {
    print(f"blank='{blank}', counter={counter}");
    return 0;
}
```

```
blank='', counter=0
```

Scope and shadowing

A variable lives inside the `{ }` block where you created it. You may reuse a name in an inner block (this is called *shadowing*), but you cannot declare the same name twice in the same block.

Chapter 6. Types

YLang has these types:

Type	What it holds	Example
<code>int</code>	Whole numbers (signed 64-bit)	<code>42</code> , <code>-7</code>
<code>float</code>	Decimal numbers (64-bit)	<code>1.25</code>
<code>bool</code>	<code>true</code> or <code>false</code>	<code>true</code>
<code>char</code>	One single byte (a letter like <code>'Y'</code>)	<code>'Y'</code>
<code>string</code>	Text. It cannot be changed after creation.	<code>"Hello"</code>
<code>void</code>	"Nothing". Only for function return types.	(none)

```
let int n = 42;
let float ratio = 1.25;
let bool ready = true;
let char initial = 'Y';
let string message = "Hello";
```

No automatic conversions

YLang never mixes types behind your back. You cannot add an `int` and a `float`:

```
let int a = 5;
let float b = 2.0;
print(a + b);
```

```
error[E2034]: Arithmetic operands must have the same numeric type.
--> file.yl:4:13
  |
4 |     print(a + b);
  |           ^
  = help: Use two int values or two float values; convert
         explicitly when needed.
```

Similarly, you cannot put text into an `int` variable:

```
let int age = "sixteen";
```

```
error[E2001]: Variable initializer type mismatch.
...
  = help: 'age' has type int, but its initializer has type string.
```

To change a value from one type to another, you must say so with a conversion function. See Chapter 13.

Number literals

Besides normal decimal numbers, YLang accepts:

- **Hexadecimal** integers that start with `0x`, such as `0xFF` (which is 255).
- **Scientific notation** for floats, such as `1e3` (which is 1000) or `2.5e-4` (which is 0.00025).

```
print(0xFF, 0x10, 1e3);
```

```
255 16 1000
```

Other type rules

- `%` (remainder) works only on `int`.
- Conditions in `if`, `while`, and `for` must be a `bool`. You cannot write `if (1)`.
- Strings can be compared with `==` and `!=`, and also ordered with `<`, `<=`, `>`, and `>=`. The order goes byte by byte, so every capital letter comes before every small letter: `"Z" < "a"` is `true`.
- A `char` is one byte, so it is meant for plain ASCII letters, not for emoji or Arabic letters.

Chapter 7. Printing and F-Strings

print

`print` shows values and then starts a new line. If you give it several values, it separates them with one space:

```
print("answer:", 42);
print("odd", 7, 1.5, true, 'x');
```

```
answer: 42
odd 7 1.5 true x
```

You can print `int`, `float`, `bool`, `char`, and `string` values.

`print` is a **statement**, not a function. That means you write `print(x);` on its own line. You cannot use it inside another expression.

Printing floats

Computers store decimals in binary, so a few results show tiny rounding differences. This is normal in many languages:

```
print(0.1 + 0.2);
print(1.5, 100.0, 2.0 / 3.0);
```

```
0.30000000000000004
1.5 100 0.6666666666666663
```

Notice that `100.0` prints as `100`.

F-strings

Put an `f` before the opening quote and write expressions inside `{ }`:

```
let const string project = "YLang";
let int sum = 30;

print(f"Welcome to {project}!");
print(f"next={sum + 1}");
```

```
Welcome to YLang!
next=31
```

You can use variables and operators inside the braces. **You cannot call functions or assign values inside the braces.** Do the work on the line before:

```
let string kind = describe(i);    // call the function first
print(f"{i} is {kind}");         // then use the result
```

To write a literal brace, double it:

```
print(f"Progress: 100% {{complete}}");
```

```
Progress: 100% {complete}
```

Escape characters

Inside a normal string, `\n` means a new line, and `\"` means a quote character.

Chapter 8. Operators

Kind	Operators
Arithmetic	+ - * / %
Comparison	== != < <= > >=
Boolean	and or not
Negative sign	- (example: -5)
Grouping	()

Note that boolean operators are **words**, not symbols: write `and`, `or`, `not`, never `&&`, `||`, `!`.

```
let bool ok = (5 > 3) and not (2 > 4);
```

Order of operations

From strongest to weakest: unary (`-`, `not`), then `* / %`, then `+ -`, then comparisons, then `and`, then `or`. When in doubt, use parentheses.

Assignment is a statement

`x = 5;` is a full statement. You cannot hide an assignment inside another expression.

Shortcuts: `+=` and `++`

Two shortcuts make common updates shorter:

```
let int count = 1;
count += 4;      // same as: count = count + 4;
count++;         // same as: count = count + 1;
print(count);
```

6

Rules, all tested:

- `+=` works on `int` and on `float` variables, and also on array elements such as `values[1] += 5;`.
- `++` works only on plain `int` variables. `f++` on a `float` is an error, and so is `values[0]++`.
- Both are **statements**. `let int b = a++;` is rejected with `E2023`.
- They cannot be used on constants.
- The other shortcuts (`-=`, `*=`, `/=`, and `--`) do not exist. Write `a = a - 1;` instead.

Chapter 9. Making Decisions with if

```
function main() -> int {
    let int level = 12;
    if (level < 10) {
        print("beginner");
    } else if (level < 20) {
        print("intermediate");
    } else if (level < 30) {
        print("advanced");
    } else {
        print("expert");
    }
    return 0;
}
```

intermediate

Rules:

- The condition goes in parentheses ().
- The body always goes in curly braces { }, even for one line.
- The condition must be a `bool`.
- `else if` and `else` are optional.

Chapter 10. Repeating Things: loop, while, for

YLang has three kinds of loops.

loop()

`loop()` repeats forever, until you stop it with `break`:

```
function main() -> int {
  let int count = 0;
  loop() {
    count = count + 1;
    print(f"Count: {count}");
    if (count >= 3) {
      break;
    }
  }
  return 0;
}
```

```
Count: 1
Count: 2
Count: 3
```

while

`while` repeats as long as a condition is true:

```
let int count = 0;
while (count < 3) {
  count = count + 1;
  print(f"Count: {count}");
}
```

```
Count: 1
Count: 2
Count: 3
```

The condition must be a `bool`. This is rejected:

```
while (42) {
  break;
}
```

```
error[E2051]: while condition must have type bool.
--> tests/loop-condition-error.yl:2:12
  |
2 |   while (42) {
  |         ^^
  = help: Write a comparison such as 'while (count < limit)'.
```

for

`for` has three parts inside the parentheses, separated by semicolons: a starting step, a condition, and a step after each round.

```
for (let int i = 0; i < 3; i = i + 1) {  
    print(i);  
}
```

```
0  
1  
2
```

Read it as: "start with `i` at 0; keep going while `i < 3`; after each round, add 1 to `i`." You can write the last part as `i++` too, so `for (let int i = 0; i < 3; i++)` does the same thing.

Good to know:

- A variable created in the first part (`let int i = 0`) only exists inside the loop.
- The first part can also be a plain assignment, like `for (k = 5; k > 3; k = k - 1)`.
- You can leave out the first part, the condition, or the last step: `for (; n < 3;) { ... }`. If you leave out the condition, the loop never stops by itself.
- Loops can sit inside loops (nested loops).

break and continue

- `break;` leaves the loop immediately.
- `continue;` skips the rest of the current round and goes to the next one. In a `for` loop, the last step still runs.
- Both work only inside a loop and affect the innermost loop.

Here is a loop that adds the odd numbers and stops after 7:

```
function main() -> int {  
    let int total = 0;  
    for (let int i = 1; i <= 10; i = i + 1) {  
        if (i % 2 == 0) { continue; }  
        if (i > 7) { break; }  
        total = total + i;  
    }  
    print("odd total:", total);  
    return 0;  
}
```

```
odd total: 16
```

Nested loops, a small multiplication table:

```
for (let int r = 1; r <= 2; r = r + 1) {  
  for (let int c = 1; c <= 3; c = c + 1) {  
    print(f"{r}x{c}={r * c}");  
  }  
}
```

```
1x1=1  
1x2=2  
1x3=3  
2x1=2  
2x2=4  
2x3=6
```

Chapter 11. Functions

A function is a named, reusable piece of code. You write the parameter types and the return type:

```
function add(int a, int b) -> int {
    return a + b;
}

function main() -> int {
    print("Sum:", add(10, 20));
    return 0;
}
```

Sum: 30

The shape is:

```
function name(type param, type param) -> returnType {
    ...
}
```

Points to remember:

- Use `return value;` to send a value back.
- A function that returns `void` returns nothing and does not need `return`.
- A function that returns a value must return on every path.
- You cannot have two functions with the same name (no overloading).
- A function that returns `void` is still called as a normal statement:

```
function greet(string who) -> void {
    print(f"Hi {who}");
}

function main() -> void {
    greet("Sam");
}
```

Hi Sam

Chapter 12. Arrays

An array holds several values of the same type, in a row. In YLang, arrays have a **fixed size**: you write the values when you create the array, and YLang counts them for you.

```
let int scores[] = [10, 20, 30];
```

The empty brackets `[]` after the name mean "figure out the length from the list".

Reading and changing elements

Positions start at **0**, not 1. Use square brackets to read or change an element:

```
function main() -> int {
  let int values[] = [10, 20, 30];
  values[1] = 42;
  print("first:", values[0]);
  print("updated:", values[1]);
  print("last:", values[2]);
  return 0;
}
```

```
first: 10
updated: 42
last: 30
```

Arrays can also be global (declared outside functions).

Looping over an array

A **for** loop and the index are the natural way to visit every element:

```
let int arr[] = [4, 8, 15];
let int sum = 0;
for (let int i = 0; i < 3; i = i + 1) {
  sum = sum + arr[i];
}
print("sum", sum);
```

```
sum 27
```

You do not need to remember the size. The built-in `length(arr)` gives the number of elements, so the loop above can say `i < length(arr)`.

The length function

`length(values)` returns the number of elements as an `int`. It works only on arrays:

```
let int numbers[] = [2, 4, 6, 8];
print("Length:", length(numbers));
```

```
Length: 4
```

`length(3)` is an error (E2027: length requires an array variable).

Arrays and functions

A function can take an array. Write `[]` after the parameter name, and pass the array by its name. The function can ask for the length with `length`:

```
function sum(int values[]) -> int {
    let int i = 0;
    let int total = 0;
    while (i < length(values)) {
        total = total + values[i];
        i = i + 1;
    }
    return total;
}

function main() -> int {
    let int numbers[] = [2, 4, 6, 8];
    print("Length:", length(numbers));
    print("Sum:", sum(numbers));
    return 0;
}
```

```
Length: 4
Sum: 20
```

Two things to remember:

- **The function sees the real array, not a copy.** If it changes an element, the change is visible to the caller. In my test, a function that added 100 to `xs[0]` changed `a[0]` from 1 to 101 in `main`.
- **The types must match.** Passing a single `int` where an array is expected gives E2029 (this parameter expects an array argument). Passing a `float` array to an `int` array parameter gives E2042 (function array element type mismatch). Arrays of `string` work too.

Safety checks

Every array access is checked while the program runs. Using a position that does not exist stops the program:

```
let int values[] = [1, 2, 3];
print(values[3]);
```

```
YLang runtime error: array index out of bounds
```

A negative index is caught in the same way. The exit status is 70.

Array rules

- You must give the values when you create the array. `let int values[] = 3;` is an error (E2038: array declarations require an array literal initializer).
- All values must have the same type. `[1, true, 3]` is an error (E2025: all array elements must have the same type).
- An array needs at least one value. `[]` is an error (E2024).
- The index must be an `int`.
- You cannot print or use a whole array as one value. YLang says (E2029): an array must be indexed before it can be used as a value. Print its elements one by one.
- You **cannot return arrays** from functions.
- No nested arrays, no `const` arrays, and no resizing.

Chapter 13. Converting Between Types

YLang never converts types behind your back. When you need a different type, you ask for it with a conversion function.

Function	What it does
<code>to_float(n)</code>	Turns an <code>int</code> into a <code>float</code> .
<code>to_int(x)</code>	Turns a <code>float</code> into an <code>int</code> by cutting off the decimal part (towards zero).
<code>to_string(v)</code>	Turns an <code>int</code> , <code>float</code> , <code>bool</code> , <code>char</code> , or <code>string</code> into a <code>string</code> .
<code>string.is_int(text)</code>	<code>true</code> if the whole text is a whole number.
<code>string.parse_int(text)</code>	Turns text like <code>"-42"</code> into an <code>int</code> .

```
function main() -> int {
  let int count = 9;
  let float precise = to_float(count);
  print(math.sqrt(precise));
  print(to_int(3.9));
  print(to_string(count));
  print(to_string(2.5));
  print(to_string(true));
  print(to_string('Z'));
  print(string.is_int("123"));
  print(string.is_int("12x"));
  print(string.parse_int("-42"));
  return 0;
}
```

```
3
3
9
2.5
true
Z
true
false
-42
```

Rules and safety

- `to_float` accepts only an `int`, and `to_int` accepts only a `float`. Using the wrong one is an error:

```
error[E2045]: to_int expects a float argument.
--> tests/invalid-to-int.yl:2:17
  |
2 |     let int n = to_int(42);
  |                   ^^^^^^^
= help: Use to_int(float_value); conversion truncates toward zero
and checks range.
```

- `to_int(-3.9)` gives `-3`, not `-4`. It cuts towards zero. If you want rounding, use `math.round` first: `to_int(math.round(x))`.
- `to_int` is range-checked. A number that is too big, or "not a number", stops the program:
`YLang runtime error: to_int value is outside the signed 64-bit range.`
- `string.is_int` is strict. These are all `false`: an empty string `"`, a lone minus `"-"`, a number with a space like `" 5"`, and a number too big for 64 bits.
- `string.parse_int` accepts a leading zero (`"007"` becomes `7`).

Reading numbers safely

`string.parse_int` stops the program on bad text, so check with `string.is_int` first, especially when the text comes from a user:

```
let string line = io.read_line();
if (string.is_int(line)) {
    let int n = string.parse_int(line);
    print(n * 2);
} else {
    print("Please type a whole number.");
}
```

If you skip the check and the text is `abc`, you get: `YLang runtime error: string.parse_int received invalid or out-of-range integer text.`

Chapter 14. The Math Library

Math functions live in the built-in `math` namespace. You write `math.` and then the function name:

```
function main() -> int {
  let float root = math.sqrt(81.0);
  let float power = math.pow(2.0, 8.0);
  let float bounded = math.clamp(12.0, 0.0, 10.0);

  print("sqrt:", root);
  print("pow:", power);
  print("sin:", math.sin(0.0));
  print("clamp:", bounded);
  print("max:", math.max(3.0, 7.0));
  return 0;
}
```

```
sqrt: 9
pow: 256
sin: 0
clamp: 10
max: 7
```

The functions

Arguments	Functions
One	<code>sqrt</code> <code>sin</code> <code>cos</code> <code>tan</code> <code>asin</code> <code>acos</code> <code>atan</code> <code>sinh</code> <code>cosh</code> <code>tanh</code> <code>exp</code> <code>log</code> <code>log10</code> <code>floor</code> <code>ceil</code> <code>round</code> <code>abs</code>
Two	<code>pow</code> <code>atan2</code> <code>min</code> <code>max</code> <code>hypot</code>
Three	<code>clamp(value, low, high)</code>

Important rules

- **Every argument must be a `float`, and every result is a `float`.** YLang does not turn `9` into `9.0` for you. Write `math.sqrt(9.0)`, or convert an `int` with `to_float(n)`:

```
error[E2042]: Math functions require float arguments.
--> tests/math-type-error.yl:2:32
  |
2 |     let float root = math.sqrt(9);
  |                               ^
= help: Use a float value, for example math.sqrt(9.0). YLang does
not implicitly convert int to float.
```

- `math.clamp(value, low, high)` forces a number into a range. If `low` is bigger than `high`, the program stops with a runtime error.

- Some results follow the normal C math rules. For example, `math.sqrt(-1.0)` gives "not a number", which prints as `nan` or `-nan` depending on your system.

A few more results, so you know what to expect:

```
print(math.floor(3.7), math.round(2.5), math.abs(-4.0));  
print(math.hypot(3.0, 4.0), math.max(1.5, 2.5));
```

```
3 3 4  
5 2.5
```

Chapter 15. Strings, Paths and Input/Output

String functions

Strings use the built-in `string` namespace:

```
function main() -> int {
  let const string title = "YLang projects";
  print(string.length(title));
  print(string.contains(title, "projects"));
  print(string.starts_with(title, "YLang"));
  print(string.ends_with(title, "projects"));
  print(string.concat("Build ", "something.));
  print(string.upper("YLang strings"));
  print(string.lower("YLANG STRINGS"));
  print("apple" < "banana");
  return 0;
}
```

```
14
true
true
true
Build something.
YLANG STRINGS
ylang strings
true
```

Function	What it does	Returns
<code>string.length(s)</code>	Number of bytes in <code>s</code>	<code>int</code>
<code>string.contains(s, part)</code>	Is <code>part</code> somewhere inside <code>s</code> ?	<code>bool</code>
<code>string.starts_with(s, prefix)</code>	Does <code>s</code> begin with <code>prefix</code> ?	<code>bool</code>
<code>string.ends_with(s, suffix)</code>	Does <code>s</code> end with <code>suffix</code> ?	<code>bool</code>
<code>string.concat(a, b)</code>	Joins two strings into a new one	<code>string</code>
<code>string.replace(s, old, new)</code>	Replaces every <code>old</code> in <code>s</code> with <code>new</code>	<code>string</code>
<code>string.upper(s)</code>	Capital letters (English letters only)	<code>string</code>
<code>string.lower(s)</code>	Small letters (English letters only)	<code>string</code>

`string.length` counts **bytes**, not letters. For plain English letters that is the same thing. For Arabic letters or emoji, one letter uses several bytes, so the number will be bigger.

`string.replace` replaces non-overlapping matches, and an empty `old` text changes nothing:

```
print(string.replace("one fish, two fish", "fish", "bird"));
print(string.replace("aaaa", "aa", "b"));
print(string.replace("leave me", "", "X"));
```

```
one bird, two bird
bb
leave me
```

`string.upper` and `string.lower` work byte by byte, so they change only the English letters A to Z. They do not understand Arabic or other alphabets.

All arguments must be strings. `string.length(42)` is an error (E2044: this standard-library function requires string arguments).

Reading from the keyboard

`io.read_line()` waits for the user to type a line and press Enter. It gives back the text without the final newline. If there is no more input, it gives back an empty string.

```
function main() -> int {
  print("What is your name?");
  let string name = io.read_line();
  if (string.length(name) == 0) {
    print("No name given.");
    return 1;
  }
  print(string.concat("Hello, ", name));
  return 0;
}
```

Running it:

```
What is your name?
Sara
Hello, Sara
```

Files

Two functions work with text files:

- `io.write_file(path, content)` creates or overwrites a file. It returns `true` if it worked and `false` if it could not (for example, if the folder does not exist). It does not create missing folders.
- `io.read_file(path)` reads a whole text file into a string. If the file cannot be opened, the program **stops** with a runtime error.

```
function main() -> int {
    let bool wrote = io.write_file("name.txt", "YLang file I/O");
    print(wrote);
    let string content = io.read_file("name.txt");
    print(content);
    return 0;
}
```

```
true
YLang file I/O
```

Remember: paths are relative to the folder where you run the program, only text files are supported, and binary files are not.

Reading a missing file stops the program:

```
YLang runtime error: cannot open file for reading
```

Writing to a folder that does not exist does not crash. It returns `false`:

```
let bool ok = io.write_file("/no/such/dir/x.txt", "hi");
print(ok);
```

```
false
```

`io.file_exists(path)` returns `true` or `false`, so you can check a file before reading it and avoid the runtime error:

```
if (io.file_exists("name.txt")) {
    print(io.read_file("name.txt"));
}
```

Paths

The `path` namespace helps with file names. It only looks at the text of the path (and `path.exists` asks the system). All three need string arguments (`path.exists(42)` gives `E2044`).

```
print(path.exists("README.md"));
print(path.basename("assets/icons/preview.png"));
print(path.extension("assets/icons/preview.png"));
print(path.extension(".profile"));
```

```
true
preview.png
png
```

- `path.exists(p)` is `true` if something exists at that path. It can be a file or a folder.
- `path.basename(p)` gives the last part: `"assets/icons/preview.png"` becomes `"preview.png"`. A trailing slash is ignored: `"/a/b/"` gives `"b"`.

- `path.extension(p)` gives the text after the last dot, without the dot: `"png"`. A name that only starts with a dot (like `.profile`) has no extension, so you get an empty string. A name with no dot also gives an empty string.

Opening a file with another program

`image.open(path)` asks your computer to open the file with its default program (for example, an image viewer). It uses `xdg-open` on Linux and `open` on macOS. It returns `true` if the launcher started, and `false` if not.

```
if (image.open("photo.png")) {  
    print("Requested the system default viewer.");  
}
```

Be clear about what it does not do: it does not show the image inside YLang and it does not understand image formats. It needs a desktop session. I tested only the failure path (a missing file returns `false`), because my test machine has no desktop.

A memory note

`string.concat`, `string.upper`, `string.lower`, and f-strings keep their results in memory until the program ends. A program that does this millions of times inside a loop will slowly use more memory.

Chapter 16. Reading Errors and Using fix

A YLang error looks like this:

```
error[E1002]: Expected ';' after variable declaration.
--> tests/missing-semicolon.yl:3:5
  |
3 |     print(age);
  |     ^^^^^
= help: The declaration may be missing ';' before this token.
```

How to read it:

- 1 **E1002** is the error code. Codes starting with **E1** are syntax (grammar) errors. Codes starting with **E2** are meaning errors such as wrong types or unknown names.
- 2 **tests/missing-semicolon.yl:3:5** means file, line 3, column 5.
- 3 The line of code is shown, and **^** points at the problem.
- 4 **help** gives a hint.

A useful tip: the arrow shows where YLang **noticed** the problem. In the example above, the missing **;** is actually at the end of the **previous** line (**let int age = 16**), and YLang noticed it when it reached **print**. So if the arrow looks wrong, check the line above.

YLang can report several errors at once. Here is a file with two mistakes:

```
function main() -> int {
    let int age = "sixteen";
    prtn(age);
    return 0;
}
```

```
error[E2001]: Variable initializer type mismatch.
--> examples/type-error.yl:2:19
  |
2 |     let int age = "sixteen";
  |                  ^^^^^^^^^
= help: 'age' has type int, but its initializer has type string.

error[E2040]: Unknown function name; 'print' is a built-in
statement, not a function.
--> examples/type-error.yl:3:5
  |
3 |     prtn(age);
  |     ^^^^^
= help: Did you mean 'print(value);'? Use print as a statement.
```

At the end, YLang tells you the result:

```
ylang: build failed with 2 error(s); no executable was produced.
```

The error code list

The file `docs/error-codes.md` in the YLang folder explains the error codes. It does not list every code yet, so if you see a code that is not there, read the message and the `help` line. Two useful groups: codes starting with `E1` are about spelling and grammar, and codes starting with `E2` are about names and types.

The fix command

There is one typo YLang can fix for you: writing `pritrn(...)` instead of `print(...)`.

```
ylang fix broken.yml -o fixed.yml
ylang check fixed.yml
```

`fix` never changes your original file. It writes a new file, and you must choose the name with `-o`. It fixes only this one typo and nothing else. Always run `check` on the new file afterward. For example, after fixing `examples/type-error.yml`, the `pritrn` is repaired, but the type error on line 2 is still there, because `fix` does not guess about that.

Chapter 17. Runtime Errors

Some mistakes only appear while the program is running. YLang adds checks for these. When one happens, the program prints a message and stops with exit status **70**.

Problem	Message
Integer division by zero	YLang runtime error: integer division by zero
Decimal division by zero	YLang runtime error: floating-point division by zero
Integer too big or too small	YLang runtime error: integer overflow in addition
Array position does not exist	YLang runtime error: array index out of bounds
<code>math.clamp</code> with low above high	YLang runtime error: math.clamp lower bound exceeds upper bound
<code>io.read_file</code> cannot open the file	YLang runtime error: cannot open file for reading
<code>to_int</code> value too big, or not a number	YLang runtime error: to_int value is outside the signed 64-bit range
<code>string.parse_int</code> with bad text	YLang runtime error: string.parse_int received invalid or out-of-range integer text

Division by zero:

```
function main() -> int {  
  let int x = 4 / 0;  
  print(x);  
  return 0;  
}
```

YLang runtime error: integer division by zero

Integer overflow. An `int` holds numbers up to 9223372036854775807. Going past that is caught:

```
function main() -> int {  
  let int max = 9223372036854775807;  
  let int overflow = max + 1;  
  print(overflow);  
  return 0;  
}
```

YLang runtime error: integer overflow in addition

The smallest `int` is fine to use: `-9223372036854775808` works.

Your own programs can also choose an exit status: `return 1;` from `main` tells the system that something went wrong.

Chapter 18. A Complete Program

Here is a program that uses arrays in functions, `length`, `for`, `while`, `+=`, `++`, type conversions, and safe number reading. Save it as `stats.yl`:

```
// Reads whole numbers from the keyboard and reports statistics.
let const int MAX = 5;

function average(int xs[]) -> float {
  let int total = 0;
  for (let int i = 0; i < length(xs); i++) {
    total += xs[i];
  }
  return to_float(total) / to_float(length(xs));
}

function largest(int xs[]) -> int {
  let int best = xs[0];
  for (let int i = 1; i < length(xs); i++) {
    if (xs[i] > best) {
      best = xs[i];
    }
  }
  return best;
}

function main() -> int {
  let int values[] = [0, 0, 0, 0, 0];
  let int count = 0;

  print(f"Enter {MAX} whole numbers, one per line:");
  while (count < MAX) {
    let string line = io.read_line();
    if (not string.is_int(line)) {
      print(string.concat("Not a whole number: ", line));
      return 1;
    }
    values[count] = string.parse_int(line);
    count++;
  }

  print("Average:", average(values));
  print("Largest:", largest(values));
  return 0;
}
```

Build and run:

```
ylang check stats.yl
ylang build stats.yl -o stats
./stats
```

Type `4`, `8`, `15`, `16`, `22` (one per line). The output is:

```
Enter 5 whole numbers, one per line:  
Average: 13  
Largest: 22
```

If you type `abc` instead of a number, the program stops politely, with exit status 1:

```
Not a whole number: abc
```

Ideas to practice:

- Add a `smallest` function, like `largest`.
- Print the numbers in order with a loop and `length`.
- Save the average to a file with `io.write_file` and `to_string`.

Chapter 19. Optional Extras

VS Code support

The repository has a VS Code extension in `editors/vscode`. It gives you:

- Colors (syntax highlighting) for `.yl` files
- Error messages from the real `ylang check` command, shown as you type
- A "Run Code" shortcut (needs the Code Runner extension)
- Light and dark icons for `.yl` files

Install it with the script that comes with YLang. Run this from the YLang folder:

```
sh docs/install-vscode.sh
```

It copies the extension into `~/.vscode/extensions/ylang-language-support`. Then restart VS Code (or run "Developer: Reload Window") and tell it where your compiler is with the `ylang.compilerPath` setting. The file `editors/vscode/README.md` has the details. To see error messages inline, also install the Error Lens extension.

I ran the script in a test folder and confirmed that it copies the extension files. I did not open VS Code itself, so I cannot confirm how the colors and error messages look.

The GUI runtime (advanced)

YLang includes a small window-drawing library built on SDL2. **Right now it is a C library, not part of the YLang language.** That means you cannot open a window from a `.yl` file yet. The README says direct calls from `.yl` code are waiting for a foreign-function interface that is not defined yet.

What it can do today (from C code): open a window, handle the close button, clear and show frames, and draw filled rectangles. To try the C example:

```
sudo apt install libsdl2-dev pkg-config      # Debian / Ubuntu
make gui gui-example
./build/gui-example
```

(On macOS use `brew install sdl2 pkg-config` and `gmake gui gui-example`.) You need a screen to run it. I could not test this part, because SDL2 is not installed in my test environment.

Chapter 20. What YLang Does Not Have

YLang is honest about its limits. These are **not** available, and programs that use them are rejected:

- Classes and objects
- Pointers and references
- `try / catch`
- Modules and imports (each file is one whole program)
- Generics
- Returning arrays from functions, nested arrays, and `const` arrays
- The shortcuts `-=`, `*=`, `/=`, and `--`
- Opening windows from `.yl` code
- Native Windows support
- Showing images inside YLang (`image.open` only starts another program)

Be careful with the rejection: when you use something unsupported, YLang may not say "this is not supported". It can report an ordinary syntax error. If you get a strange syntax error on a line that looks right, ask yourself whether you used a feature from this list.

A few more things to know:

- `char` is one byte, not a Unicode letter. Likewise `string.length` counts bytes.
- Some string results stay in memory until the program ends. A program that creates millions of them in a loop may use a lot of memory.
- YLang programs are normal native programs with your user's permissions. The YLang authors say not to use this release for security-critical code.
- There is no defined memory-safety model. Do not expect the guarantees of a language like Rust.

Chapter 21. Quick Reference

```
// comment
let int x = 1;           // variable
let const int Y = 2;     // constant
let string s = "hi";     // string
let float f = 2.5;       // float
let bool b = true;       // bool
let char c = 'A';        // char
let int nums[] = [1, 2, 3]; // array

x = x + 1;               // assignment
x += 2;                  // shortcut
x++;                     // add one (int only)
nums[0] = 9;             // change an element
length(nums)             // number of elements
print("x is", x);        // print
print(f"x is {x}");      // f-string

if (x > 1 and b) { }     // condition
else if (not b) { }
else { }

loop() { break; }        // forever until break
while (x < 5) { x = x + 1; } // while
for (let int i = 0; i < 3; i = i + 1) { continue; }

function add(int a, int b) -> int { // function
    return a + b;
}
function sum(int xs[]) -> int { return 0; } // array parameter

to_float(x)              // int -> float
to_int(2.9)              // float -> int (cuts)
to_string(x)             // to text
string.parse_int("42")   // text -> int

math.sqrt(81.0)          // math (float only)
string.concat("a", "b")  // strings
string.replace(s, "a", "b") // replace text
path.extension("a.png")  // "png"
io.read_line()           // read a line
io.write_file("f.txt", "hi") // returns bool
io.read_file("f.txt")    // returns string
io.file_exists("f.txt")  // returns bool
```

I want to...	Command
Build the compiler	<code>make</code>
Test the compiler	<code>make test</code>
Check my program	<code>ylang check file.yl</code>
Build my program	<code>ylang build file.yl -o name</code>
See the generated C	<code>ylang emit-c file.yl -o out.c</code>

I want to...	Command
Fix <code>pritrn</code> typos	<code>ylang fix file.yl -o fixed.yl</code>
Choose speed level	<code>YLANG_OPT_LEVEL=3 ylang build ...</code>
Install VS Code support	<code>sh docs/install-vscode.sh</code>

Happy coding with YLang!